Quant Research Platform

Full Platform Documentation — module contracts & data flow (updated 2026-06)

0. Project Overview

A modular intraday futures research platform for 30+ instruments, built in Python and Streamlit. It converts raw Databento market data into enriched candle datasets, runs vectorized backtests, applies position sizing, and runs Monte-Carlo stress tests. A Rust (PyO3) extension replays L3 order-book data, and a Forex Factory scraper tags trades by news/holiday.

Extensibility: adding a transform, strategy, position sizer, or Monte-Carlo method requires only dropping a file in the right folder. No registration, no imports to update — modules are discovered and loaded dynamically.

1. Application Entry Point

The app is a single Streamlit process. Navigation is held in `st.session_state.page`; `app.py` routes to one view per page:

```
home -> data_formatter
        -> backtester
        -> analytics
        -> monte_carlo
```

Each view exposes a single `render()` function and navigates with a local `go(page)` helper. There are no cross-view imports.

2. Folder Structure

```
app.py                          Streamlit entry point / router
views/                          UI pages (render() each)
transforms/                     raw DBN -> enriched parquet (run_all plugins)
strategies/                     backtest strategies (file or package) + base.py
position_sizing/                fixed.py, kelly.py, risk_based.py
monte_carlo/                    base.py (utils), bootstrap.py
ff_data_scraper/                Forex Factory text -> ff_usd_events.parquet
heatmap_rs/                     Rust (PyO3) L3 order-book replay kernel
data/
  raw_dbn/{type}/{asset}/{dataset}/   *.dbn.zst        (immutable inputs)
  parquet/{type}/{asset}/{dataset}/   YYYY-MM-DD.parquet (working layer)
  trades/{name}.parquet               backtest outputs (flat)
  news_and_holidays/ff_usd_events.parquet
```

Structural rules

- Raw DBN files are immutable inputs — never written to by the app.
- Parquet is the working layer; reprocess by re-running a transform.
- The three-level hierarchy type / asset / dataset is enforced everywhere.
- Trades files are flat: data/trades/{name}.parquet (no sub-hierarchy).
- One parquet file per calendar day, named YYYY-MM-DD.parquet.
- Asset folder names are UPPERCASE ticker symbols (ES, NQ, GC).
- Every candle parquet is indexed by a tz-aware DatetimeIndex in America/New_York.

3. Asset Info & Tick Sizes

`ASSET_INFO` is defined in `views/backtester.py` (and mirrored in the other views). It maps each ticker to `tick_size`, `ticks_per_point` and `dollars_per_tick` (all stored explicitly). The backtester injects `tick_size` into strategy params and converts `pnl_points` to ticks via `ticks_per_point`; Analytics and Monte Carlo look up `dollars_per_tick` from the trades filename.

Asset	Tick Size	Ticks/Pt	$/Tick	Category
ES	0.25	4	12.50	Equity Index
NQ	0.25	4	5.00	Equity Index
RTY	0.10	10	5.00	Equity Index
YM	1.00	1	5.00	Equity Index
MES	0.25	4	1.25	Equity Index (Micro)
MNQ	0.25	4	0.50	Equity Index (Micro)
M2K	0.10	10	0.50	Equity Index (Micro)
MYM	1.00	1	0.50	Equity Index (Micro)
ZN	0.015625	64	15.625	Rates
ZB	0.03125	32	31.25	Rates
ZF	0.0078125	128	7.8125	Rates
ZT	0.00390625	256	7.8125	Rates
SR3	0.0025	400	6.25	Rates
CL	0.01	100	10.00	Energy
QM	0.025	40	12.50	Energy
NG	0.001	1000	10.00	Energy
RB	0.0001	10000	4.20	Energy
HO	0.0001	10000	4.20	Energy
GC	0.10	10	10.00	Metals
MGC	0.10	10	1.00	Metals
SI	0.005	200	25.00	Metals
HG	0.0005	2000	12.50	Metals
ZC	0.25	4	12.50	Grains
ZS	0.25	4	12.50	Grains
ZW	0.25	4	12.50	Grains
6E	0.00005	20000	6.25	FX
6J	0.0000005	2000000	6.25	FX
6B	0.0001	10000	6.25	FX
6C	0.00005	20000	5.00	FX
BTC	5.00	0.2	25.00	Crypto

Non-equity assets: strategies that hardcode RTH as 09:30–16:00 NY (e.g. the IVB model) are wrong for CL (pit 09:00–14:30), rates, FX and metals. Treat results on those assets with caution until per-asset session times are added.

4. Data Layer

4.1 Raw Data (Databento DBN)

Source files are `.dbn.zst` archives from Databento. Two schema families are consumed:

- **Trade / top-of-book** (TRADES, MBP-1/TBBO, OHLCV-1m): drive the 1-minute candle and big-trade transforms.
- **MBO (L3, full order book)**: every add/cancel/modify/fill event with an order id — drives the 1-second order-book transforms via the Rust kernel.

Most transforms consume a **pair of consecutive day files** (previous evening + current day) to reconstruct one complete ~23-hour Globex session. The OHLCV transform instead reads **monthly** all-asset files.

4.2 Session Construction

- Load previous + current day, indexed by ts_event (UTC).

- Trim previous day to [22:00 UTC, ...) and current to [..., 21:00 UTC) — the full Globex session (1-minute candle transforms label the session in NY time, 18:00 -> 17:00).
- Concatenate, sort, convert index to America/New_York.
- Pick the front-month contract by highest volume; flag a roll day when the back month exceeds 20% of front-month volume (still processed on the front month).

4.3 Enriched 1-Minute Candles — transforms/1m_advanced.py

Input: TBBO/MBP-1 trade+book stream (pair of days). Output: one parquet per trading day. Front month, trade date and roll flag are stored as parquet key-value metadata.

Column	Type	Description
open / high / low / close	float64	OHLC of trades in the 1-minute bar
volume	int64	total contracts traded
buy_volume / sell_volume	int32	contracts on buy / sell aggressor side
volume_delta	int32	buy_volume - sell_volume
volume_delta_pct	float64	delta as % of total volume (directional)
tick_volume	JSON str	{price: [buy_qty, sell_qty]} per price level in the bar
passive_orders	JSON str	{price: [size, order_count]} resting liquidity vs bar open

volume_delta_pct: pct = delta/volume*100 (delta as % of total bar volume, bounded to -100..+100); zero-volume bars = 0.0. **Gap filling:** zero-volume bars forward-fill OHLC from prior close, volumes = 0.

4.4 Indicators — transforms/1m_advanced_indicators.py

Reads enriched 1-minute candle parquet files; writes one indicators parquet per day with **29 columns** (filename matches the source). Standard `run_all` interface.

VWAP key	Anchor	Notes
vwap_bar_globex	18:00 NY (first bar)	typical price (H+L+C)/3 weighted by bar volume
vwap_bar_rth	09:30 NY	NaN before 09:30 / after 17:00
vwap_tick_globex	18:00 NY (first bar)	VWAP from tick_volume price distribution
vwap_tick_rth	09:30 NY	NaN before 09:30 / after 17:00

Each of the four VWAPs carries ±1/2/3 sigma bands (7 columns each = 28), plus `cumulative_delta` = cumsum(buy_volume - sell_volume), anchored at 18:00 NY and reset per file (29 columns total).

Note: earlier builds emitted an OFI-beta 'absorption_score' (beta / residual / absorption_score, 32 columns). That indicator is no longer produced.

4.5 1-Second L3 Book — transforms/1s_mbo_full_book.py & 1s_mbo_cropped.py

Input: MBO (L3) .dbn.zst pair. Output: one parquet per Globex session on a 1-second grid (end-of-second book snapshots). Trade fields are vectorized in pandas; book fields come from one sequential L3 replay in Rust (see Section 5).

Column	Type	Description
open / high / low / close	float64	trade OHLC per second
volume / buy_volume / sell_volume	int	trade volume + aggressor split per second
best_bid / best_ask	float64	top of book at end of second
aggressor_volume	JSON str	{price: [buy, sell]} traded volume by price that second
bid_depth / ask_depth	JSON str	{price: size} resting book per side

- **full_book**: writes the entire resting book each second (has a pure-Python fallback if the Rust extension is missing).

- **cropped**: keeps only levels within ±N_TICKS of the second's trade hi/lo plus far 'big' orders (size ≥ BIG_ORDER_MULT × a rolling near-book median over BASELINE_WINDOW_MIN minutes). Defaults N_TICKS=100, BIG_ORDER_MULT=2.0, BASELINE_WINDOW_MIN=30. Requires the Rust extension.

4.6 Other Transforms

- **1m_ohlcv_globex_mixed_assets.py** — plain OHLCV (open/high/low/close float64, volume int64) per asset per day, demuxed from monthly all-asset OHLCV-1m DBN files. Full 1380-bar 18:00->17:00 NY grid, OHLC forward-filled, volume 0. Covers ~30 assets.
- **ES_big_trades.py** — extracts pre-RTH and RTH large trades (size thresholds) from a TRADES stream. Note: exposes `build(prev, curr, output, ...)`, not the standard run_all interface.

5. Rust Extension — heatmap_rs (PyO3)

Sequentially replays L3 (MBO) events, maintaining an aggregated depth ladder (price -> total resting qty) and an order-id location map, and emits one end-of-second snapshot per active second. Used by the 1s_mbo transforms. Inputs are pre-encoded numpy arrays: action code (A=0 C=1 M=2 F=3 R=4 T=5), side code (B=0 A=1 N=2), price_i = round(price*1e9), sec = epoch second.

```
replay_full(acode, scode, price_i, size, oid, sec)
    -> (secs, best_bid, best_ask, bid_json, ask_json)   # full book per second

replay_cropped(acode, scode, price_i, size, oid, sec,
               n_ticks, tick_i, mult, window_sec,
               trade_sec, trade_lo, trade_hi)
    -> (secs, best_bid, best_ask, bid_json, ask_json)   # cropped + big far orders
```

Build: `maturin develop --release -m heatmap_rs/Cargo.toml` (requires Python ≥ 3.13). 1s_mbo_cropped requires the built extension; 1s_mbo_full_book falls back to pure Python. See the project memory note *heatmap-rs-build* for the maturin PATH / VIRTUAL_ENV gotchas.

6. Forex Factory Scraper — ff_data_scraper/

`ff_parser.py` parses plain-text Forex Factory USD calendar exports (pasted into `ff_data_scraper/data_usd/*.txt`) and writes `data/news_and_holidays/ff_usd_events.parquet`.

Column	Type	Description
date	Date	trading date
time	Utf8	"HH:MM" (24h) or "All Day"
event	Utf8	event name
impact	Utf8	"red" (high impact) | "grey" (holiday / bank closure)

The backtester loads this file and tags every trade with a `day_type` column — `holiday`, `high_impact`, or `normal` (holiday takes priority) — enabling news/holiday performance breakdowns.

7. Views

7.1 Home (home.py)

Navigation hub only — buttons routing to the other four views. No state.

7.2 Data Formatter (data_formatter.py)

Select an input dataset (raw_dbn or parquet) via cascading type / asset / dataset selectors, pick a transform, name the output folder. On Run it dynamically imports the transform and calls `run_all(input_folder, output_folder, skip_existing, on_progress)`, streaming progress to a log box. skip_existing enables fast incremental re-runs.

7.3 Backtester (backtester.py)

Run a strategy on a candle dataset over a date range; inspect and save trades. Looks up ASSET_INFO for the selected asset, injects tick_size (HIDDEN_PARAMS hides its widget), calls `strategy.run(folder_path,`

`start, end, params)`, computes ticks = pnl_points × ticks_per_point (display only), tags day_type from the FF data, renders a notes row when the strategy emits one, and saves to `{dataset}_{strategy}_{start}_{end}.parquet` (pnl_points only, no ticks column).

7.4 Analytics (analytics.py)

Load saved trade files, apply position sizing, compare multiple sized runs. dollars_per_tick is derived from the first token of the filename via ASSET_INFO; account_size is set once. Each instance = trades file + sizer + label. Reports $-denominated metrics: total P&L, final equity, max drawdown ($ and %), Sharpe (annualized, daily grain, sqrt 252), win rate, profit factor, skipped trades.

7.5 Monte Carlo (monte_carlo.py)

Resample the trade history into thousands of equity paths to quantify uncertainty and stress-test sizing. Sections: trades, position sizing, simulation (method + ruin + params), results. Calls `method.run(trades, sizer_module, sizer_params, params)` and renders a fan chart (1/2/3-sigma bands, sampled paths, featured + median path) and a metrics table.

8. Plugin Contracts

8.1 Transform

```
def run_all(input_folder: str, output_folder: str,
            skip_existing: bool, on_progress) -> None: ...
# on_progress(current, total, message) drives the UI log
```

8.2 Strategy

```
PARAMS = { 'name': default, ... }          # optional
def run(folder_path, start_date, end_date, params) -> pd.DataFrame: ...
# required output columns:
#   date, direction, entry_time, exit_time, entry_price, exit_price,
#   sl, tp, exit_reason, pnl_points   (+ optional notes JSON)
```

pnl_points = exit-entry (long) / entry-exit (short). Return an empty DataFrame with the right columns when there are no trades. **Never** compute a ticks column — the backtester derives it from ASSET_INFO. tick_size is auto-injected (keep it in PARAMS, list it in HIDDEN_PARAMS). A strategy may also be a package whose __init__ exposes run / PARAMS / PARAM_SECTIONS.

8.3 Position Sizer

```
PARAMS = { 'account_size': 100000.0, 'dollars_per_tick': 12.50, ... }
def apply(trades: pd.DataFrame, params: dict) -> pd.DataFrame: ...
# returns a copy + columns: size, trade_pnl (ticks*$_per_tick*size),
#   equity (account_size + trade_pnl.cumsum())
# skipped trades (size rounds to 0) -> result.attrs['skipped_trades']
```

8.4 Monte Carlo

```
PARAMS = { 'n_paths': 1000, 'seed': 42, ... }
def run(trades, sizer_module, sizer_params, params) -> dict: ...
# dict must include: equity_matrix (n_paths, n_trades+1), n_trades, method

# monte_carlo/base.py helpers:
#   build_equity_curve(trades, sizer_module, sizer_params) -> np.ndarray
#   run_paths(trades, sizer_module, sizer_params, resample_fn, n_paths, seed)
```

bootstrap.py resamples trades WITH replacement (models edge uncertainty). Params: n_paths (1000), n_trades (0 = match file length), seed (42).

9. Volume Profile Algorithm (peak-based)

Used by the IVB strategy. Produces tighter value areas than the old max-volume POC on bimodal profiles.

- Aggregate tick_volume JSON across the range into {price: total_volume}.
- Smooth with a 3-tick rolling average; find local maxima (derivative sign flip).
- Cluster peaks within 4 ticks; keep the top 5 by smoothed volume.

- Refine each cluster peak to the highest RAW-volume tick within ±3 (the POC candidate).
- Expand each candidate outward (higher-adjacent side first) until 70% of volume is captured -> VAH/VAL.
- Pick the candidate with the tightest VAH-VAL range; refine the POC to the highest raw tick inside it.

10. Shared Patterns

- **Cascading folder selector** — used by Backtester, Analytics, Monte Carlo. Returns (folder_path, asset_type, asset, dataset); widget keys encode parents to force reset.
- **Dynamic module loading** — importlib spec_from_file_location + exec_module; no __init__ or registration needed; reloaded fresh on each run.
- **Session-state persistence** — run results stored in st.session_state so they survive widget interaction; init keys at the top of render().
- **HIDDEN_PARAMS** — {"tick_size"} in backtester.py; keys here are auto-injected from ASSET_INFO instead of shown as widgets.

11. End-to-End Data Flow

```
raw_dbn/{type}/{asset}/{dataset}/                (Databento .dbn.zst)
   |  Data Formatter + transform (1m_advanced / indicators / 1s_mbo / ohlcv / big_trades)
   v
parquet/{type}/{asset}/{dataset}/YYYY-MM-DD.parquet
   |  Backtester + strategy            (+ day_type from ff_usd_events.parquet)
   v
trades/{name}.parquet   (pnl_points only)
   |                                  \
   | Analytics + sizer                 Monte Carlo + sizer
   v                                    v
sized equity curve + $ metrics         equity_matrix -> fan chart + stats
```

12. Known Limitations & Roadmap

- **Session times**: RTH hardcoded 09:30–16:00 NY in order-flow strategies — wrong for CL, rates, FX, metals.
- **Enriched data coverage**: tick_volume / passive_orders only for ES/NQ (Databento window). Other assets are OHLCV-only.
- **Tick volume**: multi-level fills recorded only at the reported price.
- **Contract rolls**: front month by volume; cross-roll backtests have a one-day discontinuity (no Panama/ratio adjustment).
- **Direction convention**: 'long'/'short' must be lowercase everywhere (exact string matching).
- **Monte Carlo bootstrap**: assumes trade independence; autocorrelated strategies get optimistic CIs.

Item	Status
pnl_points-only strategy output, backtester converts to ticks	DONE
ASSET_INFO + HIDDEN_PARAMS auto tick_size injection	DONE
Peak-based volume profile	DONE
MBO L3 1-second book transforms + Rust heatmap_rs kernel	DONE
Forex Factory news/holiday day_type tagging	DONE
IVB Model 2 — four absorption entry types + trade_type column	DONE
Per-asset session times (replace hardcoded 09:30–16:00 NY)	TODO
Trailing stop / trail to absorption level	TODO
Walk-forward validation (needs multi-year enriched data)	TODO
Prop-firm Monte Carlo (daily loss / max-drawdown constraints)	TODO